

MacraScript: A Domain-Specific Language for the Generation of Macramé Patterns

Juan Manuel Serrano Rodriguez - 20211020091, Javier Alejandro Sánchez Salamanca - 20201020167

Department of Systems Engineering
Universidad Francisco José de Caldas
Bogotá, Colombia
{jumserranor, jasanchezs}@udistrital.edu.co

Abstract—This paper presents MacraScript, a domain-specific language (DSL) designed and implemented for the creation and visualization of macramé patterns. Macramé, a textile art form based on knotting, has traditionally lacked specialized digital tools for pattern design and verification. MacraScript addresses this gap by providing a declarative syntax that allows users to define patterns programmatically. The language supports two main types of patterns: "Alpha" patterns, which are grid-based and similar to pixel art, and "Normal" patterns, which are based on staggered rows of knots. The compiler, implemented in Python, performs lexical, syntactic, and semantic analysis to generate two distinct graphical outputs: a pixel-art preview (squares for Alpha, diamonds for Normal) and a detailed knot-flow diagram, providing a comprehensive guide for the artisan. This work details the language's design, its final grammar, the compiler architecture, and the resulting visualizations, demonstrating a successful bridge between computational thinking and traditional crafts.

Index Terms—domain-specific language, macramé, compiler, pattern generation, computer graphics, digital crafts

I. INTRODUCTION

Macramé is an ancient textile art form based on creating decorative patterns through organized sequences of knots [3]. Despite its enduring popularity, the design of complex macramé patterns faces significant challenges, including a high dependency on manual planning, a propensity for errors, and a lack of specialized digital tools. Current methods rely on generic graphic design software or pixel art applications, none of which are optimized for the specific constraints of knot-based textiles.

This work introduces MacraScript, a DSL developed to overcome these limitations. MacraScript is aimed at macramé artisans, digital crafters, and educators, providing a specialized tool to create, verify, and share patterns with precision. It enables designers to define macramé patterns through code, specifying parameters such as thread count, colors, and knot sequences. The language's compiler generates two key visual outputs: a preview of the final design and a step-by-step knotting guide, effectively bridging the gap between digital design and physical creation.

Project developed for the Computer Science 3 course.

II. LANGUAGE DESIGN AND GRAMMAR

MacraScript was designed as a declarative, external DSL with a simple and intuitive syntax. The core concept is the PATTERN, which can be of type ALPHA or NORMAL.

A. Grammar

The final grammar for MacraScript is defined in a clear, readable EBNF format.

```
1 (* Program Structure *)
2 <program> ::= "START" <type> <config> <pattern>
3   > "END"
4 (* Pattern Type: ALPHA for grids, NORMAL for
5   knots *)
6 <type> ::= "ALPHA" | "NORMAL"
7 (* Configuration Block *)
8 <config> ::= <threads> [<dims>] <colors>
9 <threads> ::= "THREADS" ":" <INT>
10 <dims> ::= ("WIDTH" | "HEIGHT") ":" <INT>
11 <colors> ::= "COLORS" ":" "(" <str_list> ")"
12
13 (* Pattern Data Block *)
14 <pattern> ::= "PATTERN" "{" <row_list> "}"
15 <row_list> ::= <row>+
16 <row> ::= "ROW" ":" "(" <sequence> ")"
17
18 (* Row Content: Color indices or knot
19   directions *)
20 <sequence> ::= <alpha_seq> | <normal_seq>
21 <alpha_seq> ::= <INT> ("," <INT>)*
22 <normal_seq> ::= <DIR> ("," <DIR>)*
23 <DIR> ::= "LEFT" | "RIGHT"
```

Listing 1. Final EBNF Grammar for MacraScript

B. Alpha Patterns

Alpha patterns are defined by a grid of color indices, ideal for creating images. The user specifies the dimensions and the color index for each position in the grid.

```
1 START ALPHA
2 THREADS: 4
3 WIDTH: 4
4 HEIGHT: 4
5 COLORS: ("red", "blue")
6 PATTERN {
7   ROW: (0, 1, 0, 1)
```

```

8 ROW: (1, 0, 1, 0)
9 ROW: (0, 1, 0, 1)
10 ROW: (1, 0, 1, 0)
11 }
12 END

```

Listing 2. Example of an Alpha Pattern

C. Normal Patterns

Normal patterns are defined by rows of knot directions. The language abstracts the complexity of assigning knot pairs; the compiler automatically determines the staggered knotting sequence based on the row index.

```

1 START NORMAL
2 THREADS: 8
3 COLORS: ("red", "blue", "yellow", "green")
4 PATTERN {
5   ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
6   ROW: (LEFT, LEFT, LEFT)
7   ROW: (RIGHT, RIGHT, RIGHT, RIGHT)
8 }
9 END

```

Listing 3. Example of a Normal Pattern

III. COMPILER ARCHITECTURE AND IMPLEMENTATION

The MacraScript compiler is implemented in Python and follows a traditional three-phase architecture, orchestrated by a main GUI module.

- 1) **Lexical Analysis:** The `lexical.py` module uses regular expressions to convert the source code into a stream of tokens (e.g., `KEYWORD`, `INTEGER`, `LPAREN`).
- 2) **Syntactic Analysis:** The `sintactic.py` module employs a simple recursive descent parser to validate the token stream against the language’s grammar, building a dictionary that represents the parsed structure.
- 3) **Semantic Analysis:** The `semantic.py` module interprets the parsed structure. It validates semantic rules and, crucially, translates the high-level row-based definition of `NORMAL` patterns into a concrete list of knot instructions with their corresponding thread pairs.

IV. DESIGN AND RATIONALE

The design of MacraScript was guided by simplicity and domain-specificity.

- **Declarative Syntax:** We chose a declarative style (e.g., `THREADS: 8`) over an imperative one to be intuitive for non-programmers. Artisans define the “what,” not the “how.”
- **Abstraction in Normal Patterns:** The language’s key innovation is abstracting the staggered knot-pairing logic. The user only specifies a knot’s direction (`LEFT` or `RIGHT`), and the compiler automatically calculates which threads to pair. This significantly reduces complexity and prevents common errors.
- **Dual Visualization:** The compiler generates two outputs: a **Pixel Art Preview** for a high-level aesthetic check, and a **Knot-Flow Diagram** as a functional, step-by-step guide for crafting. This duality serves both the design and execution phases of a project.

V. RESULTS: VISUAL OUTPUTS

The compiler generates two distinct visualizations for each pattern, displayed in a Tkinter-based GUI. The system includes robust, multi-level error handling (lexical, syntactic, semantic) to provide clear feedback to the user.

A. Pixel Art Visualization

This top-view visualization provides a preview of the finished pattern.

- For **Alpha** patterns, it renders a grid of colored squares.
- For **Normal** patterns, it renders a grid of colored diamonds, where each diamond corresponds to a knot. The color of the diamond is determined by the “active” thread in the knot (the one that ties the knot), ensuring consistency with the knot-flow diagram.

B. Knot-Flow Diagram

This visualization serves as a step-by-step guide for the artisan.

- For **Alpha** patterns, it shows a snake-like flow of nodes, where each node’s color and connecting line correspond to the pattern’s color grid.
- For **Normal** patterns, it displays a detailed lattice diagram. It shows how threads cross over and under each other, with circles representing the knots. The color of each circle represents the active thread, and an arrow indicates the knot’s direction (left or right).

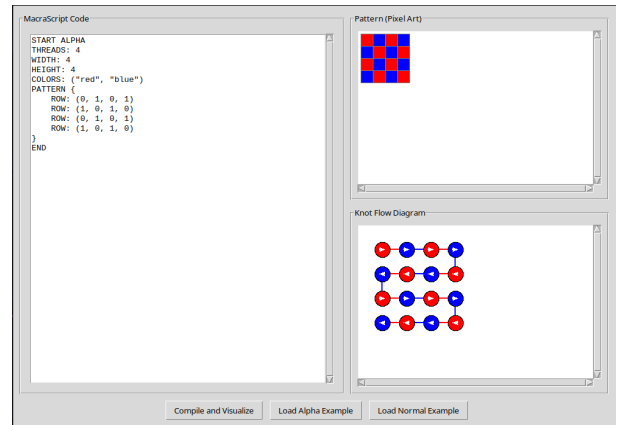


Fig. 1. Final dual visualization for an Alpha pattern.

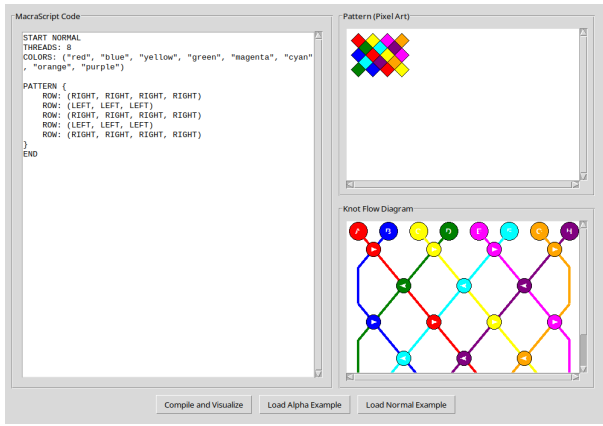


Fig. 2. Final dual visualization for a Normal pattern.

VI. CONCLUSION

MacraScript successfully demonstrates the application of DSL principles to a traditional craft. By abstracting the complexities of macramé pattern design into a simple, declarative language, it lowers the barrier to entry for creating complex designs and reduces the potential for manual error. The implementation of a full compiler that generates dual, coherent visual outputs—complemented by clear error reporting—provides a powerful and user-friendly tool for macramé artisans. Future work could involve expanding the library of knots, developing more advanced visualization techniques including interactive diagrams, and building a community platform for sharing patterns.

REFERENCES

- [1] Fowler, M. (2010). Domain-Specific Languages. Addison-Wesley Professional.
- [2] Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys (CSUR)*, 37(4), 316-344.
- [3] Karner, C. (2005). *Macramé: The craft of knotting*. Schiffer Publishing.